Bounded Memory and Bounded Work for Scoped Symbol Tables: Work in Progress

Shubhi Singh, Adhyan Jain, Akshith Venkatataramana, and Arjuman
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
[AUTHOR: confirm affiliations; add e-mail addresses]

***Abstract*—A compiler that runs inside a fixed memory budget needs a symbol table that is bounded in advance, fails in a defined way, and cannot be slowed down by hostile identifiers. We study a scoped symbol table that keeps its index, interned names and an undo stack in one caller-supplied buffer and leaves its state unchanged when an operation is refused. On symbol-table events logged by a C compiler for eight projects it needs a buffer 16.4 to 17.9 percent smaller than a conventional single-buffer table, with replay time within 10.7 percent and no established difference on most traces. Because declarations are removed in reverse order of creation, the table always equals a freshly built one, which lets us bound how often a limit on index probes refuses an honest name. Identifiers that collide for every hash seed slow declarations 300-fold; a keyed hash removes the effect.**

***Index Terms*—compilers, embedded software, hash tables, linear probing, memory management, symbol tables**

I. Problem

A scoped symbol table maps identifiers to declarations and forgets a block's declarations when the block ends. The textbook implementation is a hash table with an undo stack [1]; LLVM's `ScopedHashTable` [2] and the Tiny C Compiler [3] follow it. These tables allocate per symbol, leave their state undefined when an allocation fails, and use unkeyed hashes. For a compiler front end that must run in tens of kilobytes on source text it cannot trust, we ask for more: (R1) all state in one fixed buffer; (R2) an operation that does not fit is refused and changes nothing; (R3) a constant bound on the index slots one call examines; (R4) a negligible chance that this bound refuses an honest name. This paper reports what meeting R1 to R4 costs. The algorithm is the textbook one; what we contribute is the construction around it, two properties that follow from it, and a measurement against a baseline that shares its constraints.

II. Design

STRATA keeps everything in one array of 32-bit words (Fig. 1). The index is a linear-probing table of slots holding an 8-bit fingerprint and a record reference, at most three quarters full. There is one record per distinct visible name; it holds the value of the *innermost* declaration, so a lookup ends in the record it finds whatever the shadowing depth. Names made of identifier characters are packed five to a word. A declaration of a new name pushes one undo word, its index slot; a declaration that hides an outer one pushes three words, what it overwrote.

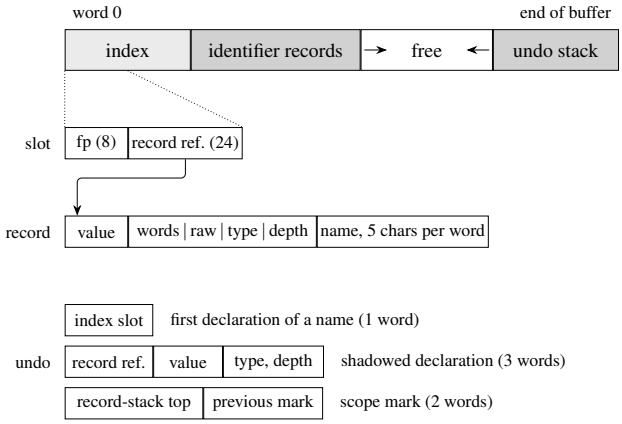


Fig. 1. One buffer of 32-bit words: index, identifier records and undo stack.

Refusal. Every operation searches, reserves, and only then writes, so a refusal (buffer full, nesting limit, invalid name, probe limit) leaves every word unchanged.

Scope exit without tombstones. In linear probing without displacement, the slots between a key's home and its position were occupied by older keys when it was inserted. The newest key therefore lies on no other key's path and can be removed by emptying its slot. Scope exit removes declarations newest first, so it writes no tombstone and moves no key.

Canonical state. It follows by induction that after any sequence of operations the table is, word for word, a fresh table into which only the live declarations were made. The table does not age, and at every moment its index is that of insertion-only linear probing.

Bounded mode. With an index of fixed size and a limit of L probes, a new name is inserted only if an empty slot lies among the first L slots of its path, and is refused otherwise; lookups stop after L slots. Dictionaries that bound probes by moving keys, such as cuckoo [4] and hopscotch hashing [5], would break the removal rule above. Because the index is an insertion-only table, the standard interval argument [6] applies: for n names with independent uniform home slots in m slots, $\alpha = n/m$ and $\rho = \alpha e^{1-\alpha}$,

$$\Pr[\text{a new name is refused}] \;\le\; \rho^{L}/(1-\rho). \qquad (1)$$

An optional keyed hash, HalfSipHash with one and three rounds [7], [8], makes the assumption hold for names chosen

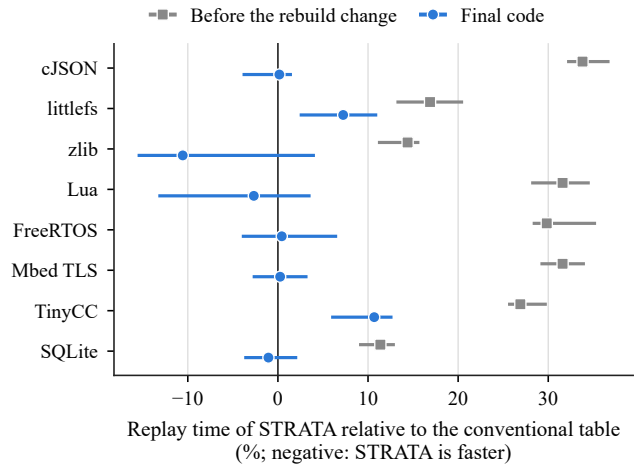


Fig. 2. Replay time of STRATA relative to the conventional table before and after the rebuild change (median of paired ratios, 95 percent interval).

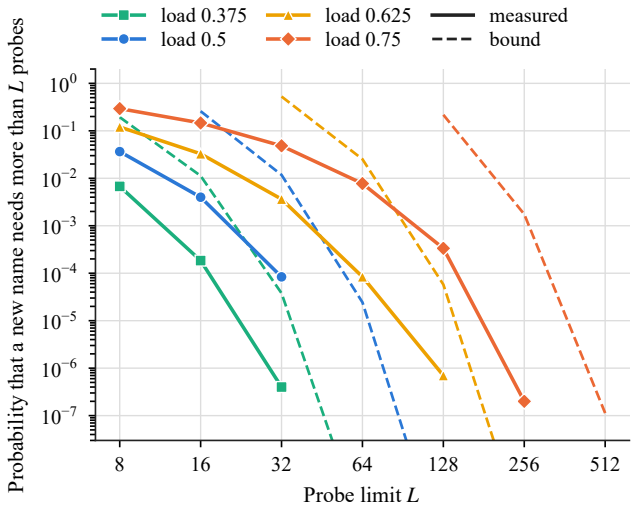


Fig. 3. Probability that inserting a fresh name examines more than L slots (keyed hash, 65 536 slots, 10 000 000 samples per load) and the bound (1).

by someone who does not know the key.

III. Evaluation

Method. We logged the symbol-table calls of the Tiny C Compiler [3] on eight open-source C projects (1 667 499 events) and replay them on 13 tables, checking every result against an independent reference model. The main baseline is a conventional chained table in one buffer with the same refusal discipline, written to be as strong as we could make it; METIS-X [9] is the multi-allocation predecessor of this work. Timing uses 31 rounds in which every table replays each trace once in rotating order; we report the median per-round ratio with a 95 percent bootstrap interval. All measurements are from one laptop.

Memory and time. All tables agree with the reference model on every trace. Table I gives the result: STRATA needs a fixed buffer 16.4 to 17.9 percent smaller than the conventional table, 27 to 36 bytes per live declaration. In time it is faster on 0 traces, slower on 2, and not distinguishable on 6. Against METIS-X it is faster on 8 of 8, but so is the conventional table (14.6 to 34.8 percent): most of that gain comes from one allocation-free buffer, not from the compact layout.

A penalty that was an artifact. An earlier version was 11.4 to 33.8 percent slower than the conventional table on every trace. Per-operation latencies placed the difference in declarations, and the cause was rebuild work: the index was rebuilt whenever the buffer grew, although it does not move; every rebuild hashed each name twice; and a new name walked its probe path twice. Removing these left memory and all results unchanged and the two tables within about ten percent of each other (Fig. 2).

Inside a compiler. With chibicc’s variable scopes on STRATA, the generated assembly is identical on 158 of 158 translation units and whole-project compile time changes by between -3.7 and $+2.2$ percent: in a hosted compiler the table does not matter.

Hostile identifiers. The default hash is a MurmurHash3-style mixer. Following Aumasson and Bernstein [7], we build 32 768 valid C identifiers that collide for every seed, restricting the construction to words that are packings of identifier characters. Declaring them takes 1954 ms against 6.5 ms for ordinary names of the same length, with quadratic growth. Under the keyed hash the ratio is 1.2; the keyed hash costs 20.2 to 32.2 percent of replay time on the compiler traces.

The probe limit. Replaying 15 compiler traces in bounded mode, a limit of 64 refuses none of 719 833 declarations. That observation does not generalize. Simulating insertions with the table’s own hash (Fig. 3), the bound was never exceeded in 168 configurations, and at the load limit of $3/4$, 0.77 percent of fresh names need more than 64 probes. The bound supports a limit of 512 (at most 1.1×10^{-7} per declaration; none seen in 10 000 000 samples).

Evidence for the guarantees. 2 732 358 randomized differential checks under sanitizers; every sequence of nine operations over four names in eight small configurations (80 621 568 sequences); 2.5 million fuzzing executions of the final code; and a mutation check in which the fuzzer detects all 9 injected defects. On an emulated Cortex-M3 with 64 KiB of memory and no heap, five traces replay with host-identical results in 2 715 bytes of code.

IV. Limitations and Next Steps

The measurements are replays on one machine; the microcontroller is emulated; the workloads are C only; the baselines other than METIS-X are our own; the two properties are argued, tested and enumerated but not machine-checked; the bound assumes an ideal hash; per-trace timing verdicts vary between runs; and the literature search was limited. The collision construction is a known technique. Next: a compiler that actually runs on the table on a physical device under a budget, a checked proof, and workloads from other languages.

TABLE I
SMALLEST FIXED BUFFER (KIB) AND REPLAY TIME OF STRATA RELATIVE TO THE CONVENTIONAL SINGLE-BUFFER TABLE AND TO METIS-X (PERCENT, 95 PERCENT INTERVAL; NEGATIVE IS FASTER).

Project	STRATA buffer	Conventional buffer	Saving (%)	Time vs. conventional table	Time vs. METIS-X
cJSON	30.0	35.9	16.4	+0.2 [−3.9, +1.6]	−22.8 [−27.5, −19.1]
littlefs	20.0	24.2	17.5	+7.2 [+2.4, +11.0]	−24.5 [−27.2, −18.1]
zlib	15.8	18.9	16.6	−10.6 [−15.6, +4.1]	−29.9 [−32.8, −27.4]
Lua	35.9	43.4	17.3	−2.7 [−13.3, +3.6]	−30.9 [−35.3, −29.2]
FreeRTOS	44.0	53.6	17.9	+0.4 [−4.0, +6.6]	−33.9 [−39.0, −31.3]
Mbed TLS	80.0	96.9	17.4	+0.2 [−2.8, +3.3]	−36.7 [−39.6, −32.4]
TinyCC	156.5	189.8	17.5	+10.7 [+5.9, +12.7]	−20.4 [−25.7, −18.7]
SQLite	140.4	169.4	17.2	−1.1 [−3.7, +2.1]	−15.9 [−20.7, −6.3]

ACKNOWLEDGMENT

The authors thank Bhuvaneswari M. for guiding the project. The implementation, experiments, analysis and text were produced with the assistance of an artificial-intelligence coding assistant (Claude, Anthropic) at the direction of the first author. **[AUTHOR: review, verify and adapt this statement to the venue's policy.]** The artifact is public [10].

REFERENCES

[1] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[2] LLVM Project, "ScopedHashTable.h, LLVM ADT library," [Online]. Available: https://llvm.org/doxygen/ScopedHashTable_8h_source.html. Accessed: Oct. 13, 2026.

[3] F. Bellard and TinyCC contributors, "Tiny C Compiler, commit 43c7708," [Online]. Available: https://repo.or.cz/tinycc.git. Accessed: Oct. 11, 2026.

[4] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[5] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 350–364.

[6] M. Thorup, "Linear probing with 5-independent hashing," arXiv:1509.04549, 2015, [Online]. Available: https://arxiv.org/abs/1509.04549.

[7] J.-P. Aumasson and D. J. Bernstein, "SipHash: A fast short-input PRF," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.

[8] J. A. Donenfeld, "SipHash – a short input PRF," Linux kernel documentation, [Online]. Available: https://docs.kernel.org/security/siphash.html. Accessed: Oct. 13, 2026.

[9] A. Jain and S. Singh, "METIS: Symbol-table research repository, commit 7b3dacf," 2026, [Online]. Available: https://github.com/adhyan-jain/Metis. Accessed: Oct. 11, 2026.

[10] S. Singh, A. Jain, A. Venkatataramana, and Arjuman, "STRATA: Source code, traces, and measurements," 2026, [Online]. Available: https://github.com/shubhisingh1510/cdesproject.